

Documentation Technique

Cluster K3S sur Debian Trixie

Projet B2 Cybersecurite - Juin 2026

Infrastructure : 3 VMs Debian Trixie | 1 Master + 2 Workers

Technologie : K3S (Kubernetes léger)

Reseau : 192.168.1.0/24 | kubes-01, kubes-02, kubes-03

Applications : Nginx, Apache HTTPD, MariaDB

Outils : kubectl, Helm, nftables

Table des matieres

- JOB 01** — Preparation et Configuration Debian Trixie
- JOB 02** — Installation K3S (Master + Workers)
- JOB 03** — Deployer les Applications
- JOB 04** — Haute Disponibilite (Replicas & Self-Healing)
- JOB 05** — Stockage Persistant (PV & PVC)
- JOB 06** — ConfigMaps (Configuration Externalisee)
- JOB 07** — Secrets (Donnees Sensibles)
- JOB 08** — RBAC & Securite (Controle d'Acces)
- JOB 09** — Helm & Industrialisation
- JOB 10** — Comparaison K3S vs Docker vs Docker Swarm
- ANNEXES** — Commandes, Depannage et Memos

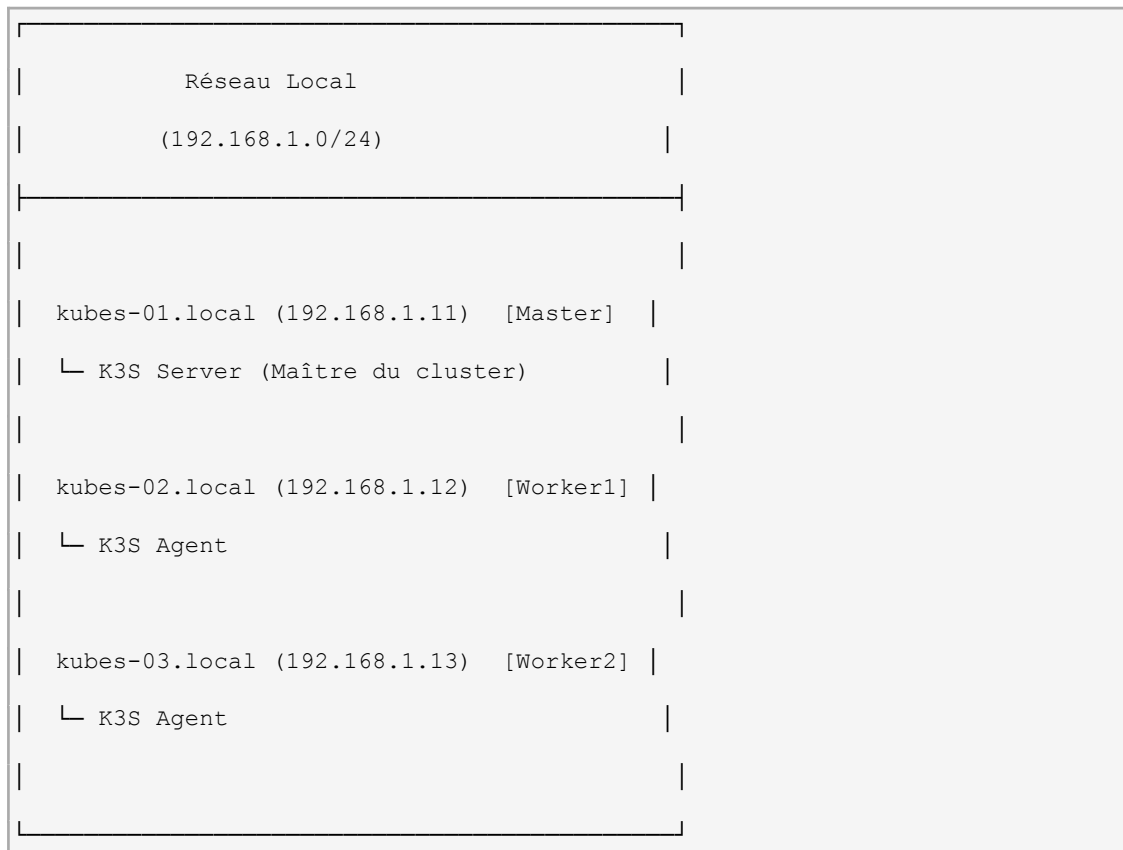
JOB 01 — Preparation & Configuration Debian Trixie

Objectif : Préparer 3 VMs Debian Trixie pour installer K3S. Configurer le réseau, le firewall (nftables), les kernel modules et les paramètres systèmes.

Durée estimée : 30 minutes

Ressources : 3 VMs avec 2 CPU min, 2 Go RAM min, 20 Go disque

Schéma du Réseau



Checklist de Préparation

1. Configuration des VMs

- ☐ 3 VMs Debian Trixie déployées
- ☐ Hostnames configurés : kubes-01, kubes-02, kubes-03
- ☐ Adresses IP statiques (192.168.1.11, 12, 13)
- ☐ Accès SSH sans mot de passe (optionnel mais recommandé)

2. Prérequis Système

- ☐ Mises à jour système (apt update && apt upgrade)
- ☐ Kernel modules chargés : br_netfilter, overlay
- ☐ Sysctl configurés : IP forwarding, bridge netfilter
- ☐ nftables configuré et persistant
- ☐ Swap désactivé

3. Vérification Réseau

- [] Ping entre les 3 VMs OK
- [] DNS résolvant les noms .local
- [] Pas de firewall bloquant les ports K3S (6443, 10250)

Configuration Détaillée

Étape 1 : Mise à Jour du Système

```
# Sur chaque VM

sudo apt update

sudo apt upgrade -y

sudo apt install -y \

    curl wget git vim htop nftables sysctl \

    ca-certificates apt-transport-https
```

Vérification :

```
uname -r          # Version du kernel

cat /etc/os-release | grep VERSION
```

Étape 2 : Configuration des Hostnames

```
# Sur kubes-01

sudo hostnamectl set-hostname kubes-01.local


# Sur kubes-02

sudo hostnamectl set-hostname kubes-02.local


# Sur kubes-03

sudo hostnamectl set-hostname kubes-03.local
```

Éditer /etc/hosts sur chaque VM :

```
sudo nano /etc/hosts
```

Ajouter les trois lignes :

```
192.168.1.11 kubes-01.local kubes-01

192.168.1.12 kubes-02.local kubes-02

192.168.1.13 kubes-03.local kubes-03
```

Vérification :

```
hostname
```

```
cat /etc/hosts  
  
ping kubes-01.local
```

Étape 3 : Configuration IP Statique (Debian Trixie)

Éditer /etc/network/interfaces ou utiliser netplan selon votre setup.

Avec netplan (moderne) :

```
sudo nano /etc/netplan/01-netcfg.yaml
```

Exemple pour kubes-01 :

```
network:  
  
  version: 2  
  
  renderer: networkd  
  
  ethernets:  
  
    eth0:  
  
      dhcp4: false  
  
      addresses:  
  
        - 192.168.1.11/24  
  
      gateway4: 192.168.1.1  
  
      nameservers:  
  
        addresses: [8.8.8.8, 8.8.4.4]
```

Appliquer :

```
sudo netplan apply  
  
ip a # Vérifier l'adresse
```

Étape 4 : Charger les Kernel Modules

Vérifier si déjà chargés :

```
lsmod | grep -E 'br_netfilter|overlay'
```

Charger les modules :

```
sudo modprobe br_netfilter  
  
sudo modprobe overlay
```

Persister au démarrage :

```
echo "br_netfilter" | sudo tee /etc/modules-load.d/k3s.conf  
  
echo "overlay" | sudo tee -a /etc/modules-load.d/k3s.conf
```

```
# Vérifier  
cat /etc/modules-load.d/k3s.conf
```

Étape 5 : Configuration Sysctl pour K3S

```
sudo nano /etc/sysctl.d/99-k3s.conf
```

Ajouter :

```
# IP Forwarding (essentiel pour le routing des pods)  
net.ipv4.ip_forward = 1  
net.ipv6.conf.all.forwarding = 1  
  
# Bridge netfilter (permet à iptables/nftables de voir le trafic des bridges)  
net.bridge.bridge-nf-call-iptables = 1  
net.bridge.bridge-nf-call-ip6tables = 1  
  
# Optimisations réseau K3S  
net.ipv4.conf.default.rp_filter = 0  
net.ipv4.conf.all.rp_filter = 0  
  
# Augmenter les connexions  
net.core.somaxconn = 1024  
net.ipv4.tcp_max_syn_backlog = 2048
```

Appliquer les changements :

```
sudo sysctl -p /etc/sysctl.d/99-k3s.conf
```

Vérifier :

```
sysctl net.ipv4.ip_forward  
sysctl net.bridge.bridge-nf-call-iptables
```

Étape 6 : Configuration Firewall (nftables - Debian Trixie)

Attention : Important : Debian Trixie n'a pas `iptables-legacy`. Nous utilisons `nftables` directement.

Éditer la configuration :

```
sudo nano /etc/nftables.conf
```

Remplacer le contenu par :

```
#!/usr/bin/nft -f

flush ruleset

# Définir les tables et chaînes
table inet filter {
    chain input {
        type filter hook input priority filter; policy accept;

        # Loopback
        iifname lo accept

        # Established connections
        ct state established,related accept

        # ICMP (ping)
        ip protocol icmp accept
        ipv6 nexthdr icmpv6 accept

        # SSH (22)
        tcp dport 22 accept

        # K3S API Server (6443)
        tcp dport 6443 accept

        # Kubelet (10250)
        tcp dport 10250 accept

        # K3S Service NodePort range (30000-32767)
        tcp dport 30000-32767 accept
        udp dport 30000-32767 accept
    }
}
```

```

# Flannel VXLAN (8472)

udp dport 8472 accept

# CoreDNS (53)

tcp dport 53 accept

udp dport 53 accept

# Drop tout le reste

reject with icmp type port-unreachable
}

chain forward {
    type filter hook forward priority filter; policy accept;
}

chain output {
    type filter hook output priority filter; policy accept;
}
}

```

Activer et démarrer nftables :

```

sudo systemctl enable nftables

sudo systemctl restart nftables

```

Vérifier les règles :

```

sudo nft list ruleset

```

Test de connectivité :

```

sudo nft list ruleset | grep dport

ping 8.8.8.8           # Internet OK?

ssh other-vm          # SSH OK?

```

Étape 7 : Désactiver le Swap

K3S/Kubernetes fonctionne mieux sans swap (le scheduler ne peut pas bien estimer les ressources).

```

# Vérifier le swap actif

```



```
free -h | grep Swap

# Désactiver temporairement
sudo swapoff -a

# Persister au démarrage
sudo sed -i '/ swap / s/^/#/' /etc/fstab

# Vérifier
free -h
```

Vérification Complète (JOB 01)

```
# Sur chaque VM, exécuter :

echo "=== HOSTNAME ==="
hostname

echo "=== IP ==="
ip a | grep 192.168

echo "=== PING INTER-NODES ==="
ping -c 2 kubes-01.local
ping -c 2 kubes-02.local
ping -c 2 kubes-03.local

echo "=== KERNEL MODULES ==="
lsmod | grep -E 'br_netfilter|overlay'

echo "=== SYSCTL IP_FORWARD ==="
sysctl net.ipv4.ip_forward

echo "=== NFTABLES RULES ==="
sudo nft list ruleset | grep -E 'dport|chain'
```

```
echo "=== SWAP ==="

free -h | grep Swap


echo "=== PORTS K3S ==="

sudo ss -tulpn | grep -E '22|6443|10250'
```

Résultat attendu :

-  Hostnames OK
-  Adresses IP statiques assignées
-  Ping inter-VMs OK
-  br_netfilter et overlay en lsmode
-  net.ipv4.ip_forward = 1
-  nftables actif et règles OK
-  Swap = 0 (désactivé)

Dépannage JOB 01

Problème	Solution
Hostnames ne pingent pas	Vérifier <code>/etc/hosts</code> et appliquer netplan
<code>br_netfilter</code> non chargé	<code>sudo modprobe br_netfilter</code> + <code>/etc/modules-load.d/</code>
nftables bloque SSH	Ajouter <code>tcp dport 22 accept</code>
<code>sysctl</code> ne change pas	Éditer <code>/etc/sysctl.d/99-k3s.conf</code> et <code>sysctl -p</code>
Swap toujours actif	Vérifier <code>/etc/fstab</code> , ajouter <code>#</code> en début de ligne swap

Prêt pour JOB 02 ?

Une fois le JOB 01 validé :

```
# Depuis une VM, vérifier:

ping kubes-01.local && ping kubes-02.local && ping kubes-03.local

# Doit retourner : 3 succès
```

→ Suivant : JOB 02 — Installation K3S

Ressources

- Debian Trixie Networking : <https://wiki.debian.org/DebianTrixie>
- nftables wiki : <https://wiki.nftables.org/>
- K3S Requirements : <https://docs.k3s.io/installation/requirements>

Notes de l'étudiant :

[À remplir lors de la réalisation]

- Adresses IP attribuées : kubes-01: ____ kubes-02: ____ kubes-03: ____
- Problèmes rencontrés : ____
- Temps d'exécution : ____

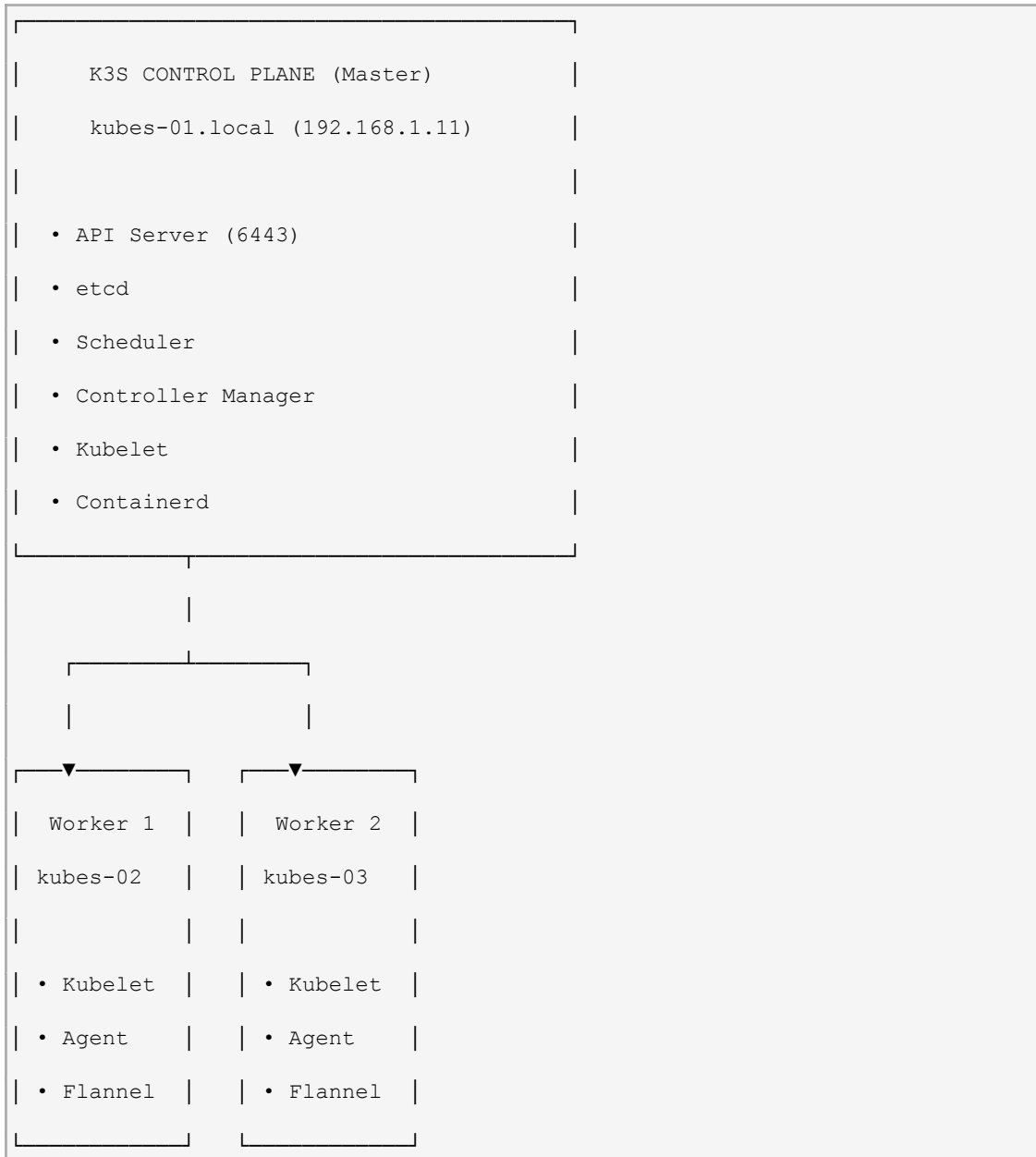
JOB 02 — Installation K3S (Master + Workers)

Objectif : Installer K3S sur les 3 nœuds et former le cluster (1 Master + 2 Workers).

Durée estimée : 20 minutes

Prérequis : JOB 01 validé (hostnames, IPs, nftables, sysctl)

Architecture du Cluster



Étapes de l'Installation

Étape 1 : Installer K3S sur le Master (kubes-01)

```
# Sur kubes-01.local (Master)

curl -sfL https://get.k3s.io | sh -
```

Que fait ce script :

- Télécharge le binaire K3S
- Configure systemd pour démarrer K3S au boot
- Crée /etc/rancher/k3s/k3s.yaml (kubeconfig)
- Démarre le service k3s

Attendre l'installation (2-5 minutes) :

```
# Vérifier que K3S est prêt

sudo systemctl status k3s


# Vérifier la version

sudo k3s --version


# Vérifier que le cluster se considère comme Ready

sudo k3s kubectl get nodes
```

Résultat attendu :

NAME	STATUS	ROLES	AGE	VERSION
kubes-01	Ready	control-plane,master	5m	v1.X.X+k3s1

Étape 2 : Récupérer le Token et l'Adresse du Master

```
# Sur kubes-01, récupérer le token du cluster

sudo cat /var/lib/rancher/k3s/server/node-token
```

Output : Quelque chose comme K1234567890abcdef....:server:xxxxxxx

Sauvegardez ce token, vous en aurez besoin pour les workers.

```
# Récupérer l'adresse IP du master

hostname -I | awk '{print $1}'
```

Output : 192.168.1.11 (ou votre adresse IP)

Étape 3 : Installer K3S sur Worker 1 (kubes-02)

```
# Sur kubes-02.local

# Remplacer TOKEN et MASTER_IP par les valeurs du master

export K3S_URL="https://192.168.1.11:6443"

export K3S_TOKEN="K1234567890abcdef....:server:xxxxxxx"


curl -sfL https://get.k3s.io | sh -
```

Attendre l'installation (2-3 minutes) :

```
sudo systemctl status k3s-agent  
  
sudo k3s-agent -v # ou sudo cat /etc/rancher/k3s/k3s-agent.env
```

Étape 4 : Installer K3S sur Worker 2 (kubes-03)

```
# Sur kubes-03.local  
  
export K3S_URL="https://192.168.1.11:6443"  
  
export K3S_TOKEN="K1234567890abcdef...:server:xxxxxxx"  
  
curl -sfL https://get.k3s.io | sh -
```

Étape 5 : Vérifier que Tous les Nœuds sont Présents

```
# Depuis le Master (kubes-01), lancer :  
  
sudo k3s kubectl get nodes -o wide
```

Résultat attendu :

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP
EXTERNAL-IP					
kubes-01	Ready	control-plane,master	10m	v1.28.x+k3s1	192.168.1.11
<none>					
kubes-02	Ready	<none>	5m	v1.28.x+k3s1	192.168.1.12
<none>					
kubes-03	Ready	<none>	4m	v1.28.x+k3s1	192.168.1.13
<none>					

Attention : Si un nœud reste `NotReady` consulter la section Dépannage.

Configuration de kubectl Local

Pour gérer le cluster depuis votre machine locale (optionnel) :

Sur le Master (kubes-01)

```
# Copier la configuration kubeconfig  
  
sudo cat /etc/rancher/k3s/k3s.yaml
```

Sur votre Machine Locale

- Copier le contenu dans ~/.kube/config
- Remplacer 127.0.0.1:6443 par 192.168.1.11:6443 (l'IP du master)

```
# Vérifier la connexion  
  
kubectl get nodes
```

Tests de Connectivité

Test 1 : Vérifier la Connexion API

```
# Depuis n'importe quel nœud ou localhost

curl -k https://192.168.1.11:6443/version
```

Doit retourner un JSON avec la version K3S.

Test 2 : Vérifier le Réseau Pod-to-Pod

```
# Lancer un pod sur le master

sudo k3s kubectl run test-pod --image=busybox --command -- sleep 3600


# Vérifier qu'il a une IP pod

sudo k3s kubectl get pods -o wide
```

Test 3 : Vérifier DNS Interne

```
# Lancer un pod et faire un DNS lookup

sudo k3s kubectl exec -it test-pod -- nslookup kubernetes.default
```

Doit résoudre 10.43.0.1 (adresse du service kubernetes).

Vérification Complète (JOB 02)

```
# Exécuter depuis le Master

echo "=== NŒUDS CLUSTER ==="

sudo k3s kubectl get nodes


echo "=== NŒUDS DÉTAILLÉS ==="

sudo k3s kubectl get nodes -o wide


echo "=== STATUS NŒUDS ==="

sudo k3s kubectl describe nodes | grep -A 2 "Name:||Ready"


echo "=== COMPOSANTS K3S ==="

sudo k3s kubectl get pods -A | grep kube


echo "=== VERSION K3S ==="

sudo k3s --version
```

```
echo "=== SERVICES K3S ==="

sudo k3s kubectl get services -A


echo "=== STOCKAGE ==="

sudo k3s kubectl get sc # StorageClasses
```

Résultat attendu :

-  3 nœuds en Ready
-  Rôles : control-plane, master pour kubes-01
-  Pods du système (coredns, flannel) en Running
-  Version K3S v1.28+
-  Services de base présents

Dépannage JOB 02

Nœud Reste `NotReady`

Symptôme :

```
kubes-02    NotReady    <none>    5m
```

Solution :

- Vérifier le token :

```
# Le token doit être identique sur master et worker

# Master:

sudo cat /var/lib/rancher/k3s/server/node-token


# Worker:

sudo cat /var/lib/rancher/k3s/agent/node-token
```

- Vérifier la connectivité réseau :

```
# Depuis le worker, tenter de joindre le master

nc -zv 192.168.1.11 6443
```

- Réinstaller le worker :

```
# Sur le worker

sudo /usr/local/bin/k3s-agent-uninstall.sh # Désinstaller proprement

# Puis réinstaller avec les bonnes variables

export K3S_URL="https://192.168.1.11:6443"

export K3S_TOKEN="votre_token"
```



```
curl -sL https://get.k3s.io | sh -
```

Erreur `connection refused` sur port 6443

Cause : Master ne répond pas sur le port API.

Solution :

```
# Sur le master, vérifier que K3S écoute

sudo ss -tulpn | grep 6443


# Redémarrer K3S

sudo systemctl restart k3s


# Vérifier les logs

sudo journalctl -u k3s -n 50
```

Pods Système en `CrashLoopBackOff`

Cause : nftables bloque le trafic inter-nœuds.

Solution : Vérifier que nftables autorise les ports K3S (voir JOB 01).

Nodes voient plusieurs IPs (IPv4 + IPv6)

Normal : K3S supporte IPv6. Si vous ne le voulez pas :

```
# Ajouter au master:

K3S_CLUSTER_INIT=true

K3S_CLUSTER_SECRET=$(openssl rand -base64 32)


# Et relancer
```

Automatisation (Optionnel)

Le script fourni scripts/02_install_k3s.sh automatise tout :

```
# Sur chaque nœud

wget https://raw.githubusercontent.com/YOUR-REPO/scripts/02_install_k3s.sh

chmod +x 02_install_k3s.sh


# Maître

sudo ./02_install_k3s.sh master


# Workers
```

```
sudo ./02_install_k3s.sh worker 192.168.1.11
K1234567890abcdef...:server:xxxxxxx
```

✓ Prêt pour JOB 03 ?

Commandes finales de validation :

```
# 1. Tous les nœuds Ready?

sudo k3s kubectl get nodes | grep Ready


# 2. Tous les pods système Running?

sudo k3s kubectl get pods -A | grep -c Running


# 3. Test de pod simple?

sudo k3s kubectl run hello --image=nginx

sudo k3s kubectl get pods

sudo k3s kubectl delete pod hello
```

→ Suivant : JOB 03 — Déployer les Applications

Ressources

- K3S Installation : <https://docs.k3s.io/installation>
- K3S Token Management :
<https://docs.k3s.io/installation/installation-requirements#operating-system-and-container-runtime-support>
- Kubernetes Nodes : <https://kubernetes.io/docs/concepts/architecture/nodes/>

Notes de l'étudiant :

```
[À remplir lors de la réalisation]

- Master token : _____

- Nœuds Ready au bout de : ____ minutes

- IPs obtenues : kubes-01: ____ kubes-02: ____ kubes-03: ____

- Problèmes rencontrés : ____
```

JOB 03 — Déployer les Applications

Objectif : Déployer 3 applications conteneurisées (Nginx, Apache, MariaDB) avec Services exposés.

Durée estimée : 25 minutes

Prérequis : JOB 02 complété (cluster opérationnel)

Applications à Déployer

App	Image	Port	Type	Objectif
Nginx	`nginx:latest`	80/443	Stateless web server	Application légère
Apache	`httpd:latest`	80	Stateless web server	Multi-conteneur
MariaDB	`mariadb:latest`	3306	Stateful database	Données persistantes

Approche Déclarative (Kubernetes-native)

Plutôt que des commandes `kubectl create`, nous utilisons des manifestes YAML (Deployments + Services).

Avantages :

-  Reproductible
-  Versionnable (Git)
-  Documenté
-  Production-ready

Étape 1 : Déployer Nginx

Créer le manifest ``nginx-deployment.yaml``

```
apiVersion: apps/v1

kind: Deployment

metadata:
  name: nginx
  labels:
    app: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
```

```
  metadata:
    labels:
      app: nginx
  spec:
    containers:
      - name: nginx
        image: nginx:latest
        ports:
          - containerPort: 80
            name: http
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            cpu: 200m
            memory: 256Mi
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: nginx-service
```

```
  labels:
```

```
    app: nginx
```

```
spec:
```

```
  type: NodePort
```

```
  selector:
```

```
    app: nginx
```

```
  ports:
```

```
    - protocol: TCP
```

```
      port: 80
```

```
      targetPort: 80
```

```
nodePort: 30080
```

Appliquer le manifest

```
# Sur le master (kubes-01)

sudo k3s kubectl apply -f nginx-deployment.yaml
```

Vérifier le déploiement

```
# Vérifier le Deployment

sudo k3s kubectl get deployment nginx

# Vérifier les Pods

sudo k3s kubectl get pods | grep nginx

# Vérifier le Service

sudo k3s kubectl get svc nginx-service
```

Tester l'accès

```
# Depuis n'importe quel nœud

curl http://192.168.1.11:30080

# Doit retourner la page par défaut Nginx (HTML)
```

Étape 2 : Déployer Apache

Créer le manifest `apache-deployment.yaml`

```
apiVersion: apps/v1

kind: Deployment

metadata:

  name: apache

  labels:

    app: apache

spec:

  replicas: 1

  selector:

    matchLabels:

      app: apache

  template:

    metadata:
```

```
  labels:
    app: apache
spec:
  containers:
  - name: httpd
    image: httpd:latest
    ports:
    - containerPort: 80
      name: http
    resources:
      requests:
        cpu: 100m
        memory: 128Mi
      limits:
        cpu: 200m
        memory: 256Mi
```

apiVersion: v1

kind: Service

metadata:

name: apache-service

labels:

app: apache

spec:

type: NodePort

selector:

app: apache

ports:

- protocol: TCP

port: 80

targetPort: 80

nodePort: 30081

Appliquer le manifest

```
sudo k3s kubectl apply -f apache-deployment.yaml
```

Tester l'accès

```
curl http://192.168.1.11:30081
```

```
# Doit retourner "It works!"
```

Étape 3 : Déployer MariaDB

Créer le manifest `mariadb-deployment.yaml`

```
apiVersion: v1

kind: ConfigMap

metadata:

  name: mariadb-init

data:

  init.sql: |

    CREATE DATABASE IF NOT EXISTS appdb;

    CREATE USER 'appuser'@'%' IDENTIFIED BY 'apppass';

    GRANT ALL PRIVILEGES ON appdb.* TO 'appuser'@'%';

    FLUSH PRIVILEGES;

---

apiVersion: apps/v1

kind: Deployment

metadata:

  name: mariadb

  labels:

    app: mariadb

spec:

  replicas: 1

  selector:

    matchLabels:

      app: mariadb

  template:

    metadata:

      labels:

        app: mariadb
```

```
spec:

  containers:

    - name: mariadb

      image: mariadb:latest

      ports:

        - containerPort: 3306

          name: mysql

      env:

        - name: MYSQL_ROOT_PASSWORD

          value: "rootpass"

        - name: MYSQL_DATABASE

          value: "appdb"

      resources:

        requests:

          cpu: 250m

          memory: 512Mi

        limits:

          cpu: 500m

          memory: 1Gi

      volumeMounts:

        - name: mariadb-init

          mountPath: /docker-entrypoint-initdb.d

  volumes:

    - name: mariadb-init

      configMap:

        name: mariadb-init

---

apiVersion: v1

kind: Service

metadata:

  name: mariadb-service

  labels:
```



```
    app: mariadb
spec:
  type: ClusterIP
  selector:
    app: mariadb
  ports:
  - protocol: TCP
    port: 3306
    targetPort: 3306
```

Appliquer le manifest

```
sudo k3s kubectl apply -f mariadb-deployment.yaml
```

Vérifier le statut

```
# Attendre que le pod soit Running (2-3 minutes)

sudo k3s kubectl get pods | grep mariadb


# Une fois Running, vérifier la base de données

sudo k3s kubectl exec -it deployment/mariadb -- mysql -u root -prootpass -e
"SHOW DATABASES;"
```

Vérification Complète (JOB 03)

```
# 1. Lister tous les Deployments

sudo k3s kubectl get deployments


# 2. Lister tous les Pods

sudo k3s kubectl get pods -o wide


# 3. Lister tous les Services

sudo k3s kubectl get svc


# 4. Vérifier les logs Nginx

sudo k3s kubectl logs deployment/nginx


# 5. Vérifier les logs Apache

sudo k3s kubectl logs deployment/apache
```

```
# 6. Vérifier les logs MariaDB

sudo k3s kubectl logs deployment/mariadb


# 7. Test HTTP Nginx

curl -I http://192.168.1.11:30080


# 8. Test HTTP Apache

curl -I http://192.168.1.11:30081


# 9. Test MariaDB

sudo k3s kubectl run -it --rm mysql-client \

  --image=mysql:latest \

  --command -- \

  mysql -h mariadb-service -u root -prootpass -e "SHOW DATABASES;"
```

Résultat attendu :

-  3 Deployments créés
-  3 Pods en Running
-  3 Services créés
-  Curl Nginx répond 200
-  Curl Apache répond 200
-  MySQL client se connecte et voit les BD

Dépannage JOB 03

Symptôme	Cause	Solution
<code>`ImagePullBackOff`</code>	Image non disponible	Vérifier la syntaxe du manifest (image:)
<code>`CrashLoopBackOff`</code>	Erreur au démarrage du conteneur	<code>`kubectl logs <pod>`</code> pour voir l'erreur
<code>`Pending`</code>	Pas de ressources / nœud pas ready	<code>`kubectl describe pod <nom>`</code>
Service injoignable (30080)	NodePort pas bien configuré	Vérifier nodePort et selector dans le Service
MariaDB très lent au démarrage	C'est normal	Attendre 1-2 minutes, c'est lourd

Organisation des Manifests

Pour garder les choses organisées, stocker tous les manifests dans un répertoire :

```
manifests/
├─ nginx-deployment.yaml
├─ apache-deployment.yaml
└─ mariadb-deployment.yaml
```

Appliquer tous les manifests à la fois :

```
sudo k3s kubectl apply -f manifests/
```

Mettre à Jour une Application

Si vous changez l'image (par exemple, passer à nginx:1.25) :

```
# Éditer le manifest
nano nginx-deployment.yaml # Changer image: nginx:latest → nginx:1.25

# Réappliquer
sudo k3s kubectl apply -f nginx-deployment.yaml

# K3S mettra à jour automatiquement les pods
sudo k3s kubectl get pods | grep nginx
```

Supprimer une Application

```
# Supprimer tout ce qui est relié
sudo k3s kubectl delete -f nginx-deployment.yaml

# Ou supprimer individuellement
sudo k3s kubectl delete deployment nginx
sudo k3s kubectl delete service nginx-service
```

✓ Prêt pour JOB 04 ?

```
# Vérifier que les 3 apps sont prêtes
sudo k3s kubectl get pods | grep -E 'nginx|apache|mariadb'

# Doit afficher 3 pods en Running
```

→ Suivant : JOB 04 — Haute Disponibilité (Replicas & Self-Healing)

Ressources

- Kubernetes Deployments :
<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

- Services : <https://kubernetes.io/docs/concepts/services-networking/service/>
- ConfigMaps : <https://kubernetes.io/docs/concepts/configuration/configmap/>

Notes de l'étudiant :

[À remplir lors de la réalisation]

- Nginx déployé: ☐ Port accessible: ☐
- Apache déployé: ☐ Port accessible: ☐
- MariaDB déployé: ☐ Connexion OK: ☐
- Problèmes: ____

JOB 04 — Haute Disponibilite (Replicas & Self-Healing)

Objectif : Implémenter la HA avec replicas, load balancing automatique et test de panne.

Durée estimée : 15 minutes

Prérequis : JOB 03 complété

Concepts HA

Concept	Rôle
Replicas	N copies d'un pod pour la redondance
ReplicaSet	Garantit que N pods tournent en permanence
Deployment	Gère les replicas et les mises à jour
Service	Load balance automatique sur les replicas
Self-healing	K3S recrée les pods qui crashent

Étape 1 : Activer les Replicas (Nginx)

Éditer nginx-deployment.yaml

```
spec:

  replicas: 3 # Passer de 1 à 3
```

Appliquer la modification

```
sudo k3s kubectl apply -f nginx-deployment.yaml
```

Vérifier la création des replicas

```
# Attendre quelques secondes

sudo k3s kubectl get pods | grep nginx


# Doit afficher 3 pods Nginx

NAME                                READY   STATUS    RESTARTS   AGE
nginx-6d4cf56db-abc12              1/1     Running   0           10s
nginx-6d4cf56db-def34              1/1     Running   0           5s
nginx-6d4cf56db-ghi56              1/1     Running   0           2s
```

Vérifier le ReplicaSet

```
sudo k3s kubectl get rs | grep nginx


# Output:

NAME                                DESIRED   CURRENT   READY   AGE
nginx-6d4cf56db                    3          3         3       1m
```

Étape 2 : Répliquer Apache et MariaDB

```
# Éditer apache-deployment.yaml

spec:

  replicas: 2


# Éditer mariadb-deployment.yaml

spec:

  replicas: 1 # Les BD stateful restent en 1 (voir JOB 05)
```

Appliquer :

```
sudo k3s kubectl apply -f apache-deployment.yaml

sudo k3s kubectl apply -f mariadb-deployment.yaml


# Vérifier

sudo k3s kubectl get pods
```

Étape 3 : Test de Self-Healing

Test 1 : Supprimer un Pod

```
# Récupérer le nom d'un pod Nginx

POD_NAME=$(sudo k3s kubectl get pods | grep nginx | head -1 | awk '{print $1}')


# Supprimer le pod

sudo k3s kubectl delete pod $POD_NAME


# Observer : K3S en recrée un automatiquement

watch sudo k3s kubectl get pods | grep nginx
```

Résultat attendu : Le pod supprimé est remplacé en quelques secondes par un nouveau.

Test 2 : Tester le Load Balancing

```
# Lancer 10 requêtes HTTP

for i in {1..10}; do

  echo "Requête $i:"

  curl http://192.168.1.11:30080 -I | grep Server

done
```

Chaque requête peut être traitée par un pod différent (transparente pour le client).

Test 3 : Simuler la Panne d'un Nœud

```
# Sur l'un des nœuds (workers)

# Identifier quel nœud exécute les pods Nginx

sudo k3s kubectl get pods -o wide | grep nginx


# Noter le nœud (kubes-02 ou kubes-03)


# Se connecter au nœud et l'arrêter :

ssh kubes-02.local

sudo shutdown -r now # Redémarrage (ou halt)
```

Observer :

- Les pods sur kubes-02 passer à Terminating puis Pending
- Le Service continuer à router vers les pods des autres nœuds
- Après ~1 min, les pods être reschedules sur kubes-03

```
# Depuis le master

watch sudo k3s kubectl get pods -o wide
```

Vérification Complète (JOB 04)

```
# 1. Vérifier les replicas

sudo k3s kubectl get deployments

# Doit afficher replicas: 3 pour nginx, 2 pour apache


# 2. Vérifier les pods

sudo k3s kubectl get pods -o wide

# Doit afficher distribués sur les 3 nœuds


# 3. Vérifier les ReplicaSets

sudo k3s kubectl get rs


# 4. Vérifier les événements

sudo k3s kubectl get events --sort-by='.lastTimestamp'
```

```
# 5. Test du Service (load balancing)

for i in {1..5}; do

    sudo k3s kubectl get svc

done
```

Stratégies de Déploiement

RollingUpdate (par défaut)

```
spec:

  strategy:

    type: RollingUpdate

    rollingUpdate:

      maxSurge: 1          # Max 1 pod supplémentaire pendant l'update

      maxUnavailable: 0   # Pas de downtime
```

Recreate (simple, avec downtime)

```
spec:

  strategy:

    type: Recreate
```

Dépannage JOB 04

Problème	Solution
Pods restent `Pending`	Pas de ressources → `kubectl describe pod`
Pods en `CrashLoopBackOff`	Erreur dans le conteneur → `kubectl logs`
Load balancing pas équilibré	Normal, c'est probabiliste. Refaire le test
Nœud reste `NotReady` après reboot	Relancer K3S: `sudo systemctl restart k3s`

✓ Prêt pour JOB 05 ?

```
# Vérifier:

sudo k3s kubectl get deployments | grep -E 'nginx|apache|mariadb'


# Doit afficher:

# nginx      3/3      3          3          5m
# apache     2/2      2          2          5m
```

→ Suivant : JOB 05 — Stockage Persistant

Notes de l'étudiant :

```
- Replicas Nginx: ☐  Apache: ☐

- Self-healing testé: ☐
```


- Nœud reboté et pods reschedules: ☐

- Observations: ____

JOB 05 — Stockage Persistant (PV & PVC)

Objectif : Configurer le stockage persistant pour MariaDB et Nginx.

Durée estimée : 20 minutes

Prérequis : JOB 04 complété

Concepts

Objet	Rôle
StorageClass	Définit les types de stockage disponibles
PersistentVolume	Ressource de stockage cluster-wide
PersistentVolumeClaim	Demande de stockage par une application
volumeMounts	Point de montage dans le conteneur

Étape 1 : Vérifier la StorageClass par Défaut

K3S fournit une StorageClass local-path par défaut :

```
sudo k3s kubectl get sc

# Output:

NAME                                PROVISIONER                RECLAIMPOLICY
VOLUMEBINDINGMODE
local-path (default)                rancher.io/local-path      Delete
WaitForFirstConsumer
```

Étape 2 : Créer une PVC pour MariaDB

Créer `mariadb-pvc.yaml`

```
apiVersion: v1

kind: PersistentVolumeClaim

metadata:
  name: mariadb-data

spec:
  accessModes:
    - ReadWriteOnce

  storageClassName: local-path

  resources:
    requests:
      storage: 5Gi

---
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mariadb-backup
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: local-path
  resources:
    requests:
      storage: 2Gi
```

Appliquer la PVC

```
sudo k3s kubectl apply -f mariadb-pvc.yaml

# Vérifier

sudo k3s kubectl get pvc
```

Étape 3 : Modifier le Deployment MariaDB

Éditer `mariadb-deployment.yaml`

Ajouter les volumes :

```
spec:
  template:
    spec:
      containers:
        - name: mariadb
          # ... reste inchangé ...
          volumeMounts:
            - name: mariadb-data
              mountPath: /var/lib/mysql
            - name: mariadb-backup
              mountPath: /backup
          volumes:
            - name: mariadb-data
```

```
    persistentVolumeClaim:
      claimName: mariadb-data
  - name: mariadb-backup
    persistentVolumeClaim:
      claimName: mariadb-backup
```

Réappliquer le manifest

```
sudo k3s kubectl apply -f mariadb-deployment.yaml

# Attendre que le pod se redémarre

sudo k3s kubectl get pods | grep mariadb
```

Vérifier le montage

```
# Se connecter au pod

sudo k3s kubectl exec -it deployment/mariadb -- ls -la /var/lib/mysql

# Doit afficher les fichiers de la BD
```

Étape 4 : Créer une PVC pour Nginx (logs)

Créer `nginx-pvc.yaml`

```
apiVersion: v1

kind: PersistentVolumeClaim

metadata:
  name: nginx-logs

spec:
  accessModes:
    - ReadWriteMany # Plusieurs pods peuvent lire/écrire

  storageClassName: local-path

  resources:
    requests:
      storage: 1Gi
```

Appliquer et modifier Nginx

```
sudo k3s kubectl apply -f nginx-pvc.yaml

# Éditer nginx-deployment.yaml

# Ajouter dans volumeMounts:
```

```
volumeMounts:

- name: logs

  mountPath: /var/log/nginx


# Dans volumes:

volumes:

- name: logs

  persistentVolumeClaim:

    claimName: nginx-logs
```

Étape 5 : Test de Persistance

Test 1 : Données persistantes après pod restart

```
# Écrire quelque chose dans MariaDB

sudo k3s kubectl exec -it deployment/mariadb -- mysql -u root -prootpass -e \

  "CREATE TABLE test (id INT, msg TEXT); \

  INSERT INTO test VALUES (1, 'K3S HA TEST');"


# Supprimer le pod

POD=$(sudo k3s kubectl get pod -l app=mariadb -o
jsonpath='{.items[0].metadata.name}')

sudo k3s kubectl delete pod $POD


# Attendre le redémarrage

sleep 10


# Vérifier que les données sont toujours là

sudo k3s kubectl exec -it deployment/mariadb -- mysql -u root -prootpass -e \

  "SELECT * FROM test;"


# Doit afficher : 1 | K3S HA TEST
```

Test 2 : Vérifier l'existence du PV

```
sudo k3s kubectl get pv

# Doit afficher:
```

# pvc-xxxxxx	5Gi	RWO	Delete	Bound
--------------	-----	-----	--------	-------

Vérification Complète (JOB 05)

```
# 1. PVCs créées
sudo k3s kubectl get pvc

# 2. PVs liés
sudo k3s kubectl get pv

# 3. Pod MariaDB avec volumeMounts
sudo k3s kubectl describe pod -l app=mariadb | grep -A 5 "Mounts:"

# 4. Test de lecture/écriture
sudo k3s kubectl exec -it deployment/mariadb -- ls -la /var/lib/mysql

# 5. Vérifier la taille utilisée
sudo k3s kubectl exec -it deployment/mariadb -- du -sh /var/lib/mysql
```

Dépannage JOB 05

Problème	Solution
PVC reste `Pending`	StorageClass pas dispo → `kubectl get sc`
Pod ne démarre pas	Impossible de monter la PVC → `kubectl describe pvc`
Permission denied sur le volume	User/permission du conteneur → vérifier securityContext

Types d'Accès (accessModes)

Mode	Signification	Cas d'usage
`ReadWriteOnce`	1 nœud peut lire/écrire	BD, stateful
`ReadOnlyMany`	Plusieurs nœuds lisent	Configs partagées
`ReadWriteMany`	Plusieurs nœuds lisent/écrivent	Logs, cache partagé

✓ Prêt pour JOB 06 ?

```
# Vérifier:

sudo k3s kubectl get pvc | grep -E 'mariadb|nginx'

# Doit afficher 3 PVCs en Bound
```

→ Suivant : JOB 06 — ConfigMaps

Notes de l'étudiant :

- PVCs créées pour MariaDB: ☐
- PVCs créées pour Nginx: ☐
- Volumes montés: ☐
- Test de persistance OK: ☐

JOB 06 — ConfigMaps (Configuration Externalisée)

Objectif : Externaliser la configuration des applications via ConfigMaps (variables, fichiers).

Durée estimée : 15 minutes

Prérequis : JOB 05 complété

Concepts

ConfigMap = clés/valeurs ou fichiers de config externalisés du code.

Avantages :

-  Configuration séparée du Deployment
-  Réutilisable par plusieurs pods
-  Mutable sans redéployer

Étape 1 : ConfigMap pour Nginx

Créer `nginx-configmap.yaml`

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  # Clés simples
  WORKER_PROCESSES: "4"
  WORKER_CONNECTIONS: "1024"

  # Fichier complet
  nginx.conf: |
    user nginx;

    worker_processes auto;

    error_log /var/log/nginx/error.log warn;

    pid /var/run/nginx.pid;

    events {
      worker_connections 1024;
    }
```



```

http {
    include /etc/nginx/mime.types;

    default_type application/octet-stream;

    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" '
        '"$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;

    sendfile on;

    tcp_nopush on;

    tcp_nodelay on;

    keepalive_timeout 65;

    types_hash_max_size 2048;

    server {

        listen 80;

        server_name _;

        location / {

            root /usr/share/nginx/html;

            index index.html;

        }

    }

}

```

Appliquer la ConfigMap

```

sudo k3s kubectl apply -f nginx-configmap.yaml

# Vérifier

sudo k3s kubectl get configmap

sudo k3s kubectl describe configmap nginx-config

```

Utiliser la ConfigMap dans Nginx Deployment

Éditer nginx-deployment.yaml :

```
spec:
  template:
    spec:
      containers:
      - name: nginx
        # ... reste ...
        env:
        - name: WORKER_PROCESSES
          valueFrom:
            configMapKeyRef:
              name: nginx-config
              key: WORKER_PROCESSES
        volumeMounts:
        - name: nginx-conf
          mountPath: /etc/nginx
        volumes:
        - name: nginx-conf
          configMap:
            name: nginx-config
```

Réappliquer :

```
sudo k3s kubectl apply -f nginx-deployment.yaml
```

Étape 2 : ConfigMap pour Apache

Créer `apache-configmap.yaml`

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: apache-config
data:
  httpd.conf: |
    ServerRoot "/usr/local/apache2"
```

```
Listen 80

LoadModule mpm_prefork_module modules/mod_mpm_prefork.so

<IfModule mpm_prefork_module>

    StartServers 10

    MinSpareServers 10

    MaxSpareServers 20

    MaxRequestWorkers 256

    MaxConnectionsPerChild 0

</IfModule>

DocumentRoot "/usr/local/apache2/htdocs"

<Directory />

    AllowOverride none

    Require all denied

</Directory>

<Directory "/usr/local/apache2/htdocs">

    AllowOverride None

    Require all granted

</Directory>

ErrorLog logs/error_log

LogLevel warn

CustomLog logs/access_log combined
```

Appliquer et utiliser :

```
sudo k3s kubectl apply -f apache-configmap.yaml

# Éditer apache-deployment.yaml pour monter le fichier

# volumeMounts:
```

```
# - name: apache-conf

#   mountPath: /usr/local/apache2/conf

# volumes:

# - name: apache-conf

#   configMap:

#     name: apache-config
```

Étape 3 : Test de ConfigMap

Vérifier le montage

```
# Vérifier que le fichier existe dans le pod Nginx

sudo k3s kubectl exec -it deployment/nginx -- cat /etc/nginx/nginx.conf

# Doit afficher la config complète
```

Mettre à jour la ConfigMap (pas de redéploiement)

```
# Éditer directement

sudo k3s kubectl edit configmap nginx-config

# Les pods ne se mettront PAS à jour automatiquement!

# Forcer un redéploiement:

sudo k3s kubectl rollout restart deployment/nginx
```

Vérification Complète (JOB 06)

```
# 1. ConfigMaps créées

sudo k3s kubectl get configmap

# 2. Vérifier le contenu

sudo k3s kubectl get configmap nginx-config -o yaml

# 3. Vérifier le montage dans le pod

sudo k3s kubectl exec -it deployment/nginx -- ls -la /etc/nginx/

# 4. Vérifier les variables d'env

sudo k3s kubectl exec -it deployment/nginx -- env | grep WORKER
```

Dépannage JOB 06

Problème	Solution
Pod ne démarre pas après apply	Configmap mal formatée → `kubectl describe pod`
ConfigMap montée mais fichier vide	Clé mal nommée dans le spec
Changement de ConfigMap pas appliqué	Redémarrer le pod: `rollout restart`

Note Importante

Attention : ConfigMaps ne sont PAS chiffrées ! Ne pas y mettre de secrets (mots de passe, clés).

→ Voir JOB 07 pour les Secrets.

✓ Prêt pour JOB 07 ?

```
sudo k3s kubectl get configmap | grep -E 'nginx|apache'

# Doit afficher 2 configmaps
```

→ Suivant : JOB 07 — Secrets (Données Sensibles)

Notes de l'étudiant :

```
- ConfigMap Nginx créée: ☐
- ConfigMap Apache créée: ☐
- Montage vérifié: ☐
```

JOB 07 — Secrets (Donnees Sensibles)

Objectif : Stocker et gérer les données sensibles (mots de passe, tokens) de manière sécurisée.

Durée estimée : 15 minutes

Prérequis : JOB 06 complété

Concepts

Secret = données sensibles chiffrées en etcd (au repos).

Types :

- Opaque : chaînes base64 (défaut)
- kubernetes.io/basic-auth : username + password
- kubernetes.io/docker-cfg : credentials Docker
- kubernetes.io/tls : certificats TLS

Étape 1 : Créer un Secret pour MariaDB

Créer ``mariadb-secret.yaml``

```
apiVersion: v1

kind: Secret

metadata:

  name: mariadb-credentials

type: Opaque

stringData:

  MYSQL_ROOT_PASSWORD: "SecureRootPass123!"

  MYSQL_USER: "appuser"

  MYSQL_PASSWORD: "AppPass456!"

  DATABASE: "appdb"
```

Ou créer avec kubectl :

```
# Créer manuellement (puis copier-coller dans YAML)

sudo k3s kubectl create secret generic mariadb-credentials \

  --from-literal=MYSQL_ROOT_PASSWORD="SecureRootPass123!" \

  --from-literal=MYSQL_USER="appuser" \

  --from-literal=MYSQL_PASSWORD="AppPass456!" \

  --from-literal=DATABASE="appdb" \

  --dry-run=client -o yaml > mariadb-secret.yaml
```

Appliquer le Secret

```
sudo k3s kubectl apply -f mariadb-secret.yaml

# Vérifier (ne montre pas les valeurs)

sudo k3s kubectl get secret

sudo k3s kubectl describe secret mariadb-credentials
```

Étape 2 : Utiliser le Secret dans MariaDB Deployment

Éditer `mariadb-deployment.yaml`

```
spec:

  template:

    spec:

      containers:

        - name: mariadb

          env:

            - name: MYSQL_ROOT_PASSWORD

              valueFrom:

                secretKeyRef:

                  name: mariadb-credentials

                  key: MYSQL_ROOT_PASSWORD

            - name: MYSQL_USER

              valueFrom:

                secretKeyRef:

                  name: mariadb-credentials

                  key: MYSQL_USER

            - name: MYSQL_PASSWORD

              valueFrom:

                secretKeyRef:

                  name: mariadb-credentials

                  key: MYSQL_PASSWORD

            - name: MYSQL_DATABASE

              valueFrom:

                secretKeyRef:
```

```
name: mariadb-credentials

key: DATABASE
```

Réappliquer le Deployment

```
sudo k3s kubectl apply -f mariadb-deployment.yaml

# Vérifier que le pod redémarre

sudo k3s kubectl get pods | grep mariadb
```

Étape 3 : Secret TLS (Certificat HTTPS)

Créer un certificat auto-signé

```
# Générer une clé privée

openssl genrsa -out tls.key 2048

# Générer un CSR

openssl req -new -key tls.key -out tls.csr \
    -subj "/CN=nginx.local"

# Auto-signer le certificat

openssl x509 -req -days 365 -in tls.csr \
    -signkey tls.key -out tls.crt
```

Créer le Secret TLS

```
sudo k3s kubectl create secret tls nginx-tls \
    --cert=tls.crt \
    --key=tls.key

# Vérifier

sudo k3s kubectl get secret nginx-tls
```

Utiliser le Secret dans Nginx (avancé)

Pour exposer HTTPS, il faudrait un Ingress (voir extensions).

Étape 4 : Test du Secret

Vérifier les variables d'environnement dans le pod

```
# Récupérer le pod MariaDB

POD=$(sudo k3s kubectl get pod -l app=mariadb -o
jsonpath='{.items[0].metadata.name}')
```



```
# Vérifier que les vars d'env sont présentes (masquées)

sudo k3s kubectl exec -it $POD -- env | grep MYSQL
```

Tester la connexion avec les credentials du Secret

```
# Connexion au conteneur MariaDB

sudo k3s kubectl exec -it $POD -- mysql -u appuser -pAppPass456! -e "SHOW
DATABASES;"

# Doit afficher les BDs de appuser (pas juste test_db)
```

Étape 5 : Décoder un Secret (audit)

Attention : Important : Les Secrets ne sont que base64 (pas chiffré par défaut) !

```
# Voir le contenu brut (base64)

sudo k3s kubectl get secret mariadb-credentials -o yaml

# Décoder une clé

sudo k3s kubectl get secret mariadb-credentials -o
jsonpath='{.data.MYSQL_ROOT_PASSWORD}' | base64 -d

# Output: SecureRootPass123!
```

Pour du chiffrement réel : voir --encryption-provider-config (étape avancée).

Vérification Complète (JOB 07)

```
# 1. Secrets créés

sudo k3s kubectl get secret

# 2. Contenu du Secret (masqué)

sudo k3s kubectl describe secret mariadb-credentials

# 3. Pod MariaDB utilise le Secret

sudo k3s kubectl exec -it deployment/mariadb -- env | grep MYSQL_

# 4. Test de connexion

sudo k3s kubectl exec -it deployment/mariadb -- \
mysql -u appuser -pAppPass456! -e "SELECT USER();"


```

Résultat attendu :

```
USER ()
appuser@%
```

Dépannage JOB 07

Problème	Solution
Mot de passe mal reconnu	Vérifier quotes et échappement dans YAML
Pod ne démarre pas	Secret non trouvé → `kubectl describe pod`
`secretKeyRef` non trouvée	Vérifier le nom et la clé du Secret

⚠ Bonnes Pratiques

- ✓ Stocker les Secrets dans un gestionnaire externe (Vault, AWS Secrets)
- ✓ Chiffrer les Secrets en etcd
- ✓ Limiter l'accès RBAC aux Secrets
- ✓ Audit les accès aux Secrets

✓ Prêt pour JOB 08 ?

```
sudo k3s kubectl get secret | grep -E 'mariadb|nginx'

# Doit afficher au moins 2 secrets
```

→ Suivant : JOB 08 — RBAC & Sécurité

Notes de l'étudiant :

```
- Secret MariaDB créé: ☐
- Secret TLS créé: ☐
- Variables env vérifiées: ☐
- Connexion MariaDB avec credentials: ☐
```

JOB 08 — RBAC & Securite

Objectif : Implémenter le contrôle d'accès basé sur les rôles (RBAC) et durcir le cluster.

Durée estimée : 20 minutes

Prérequis : JOB 07 complété

RBAC : Concepts

Objet	Rôle
ServiceAccount	Identité pour les pods/users
Role	Ensemble de permissions (namespace-scoped)
ClusterRole	Ensemble de permissions (cluster-wide)
RoleBinding	Assigne un Role à un ServiceAccount
ClusterRoleBinding	Assigne un ClusterRole à un ServiceAccount

Étape 1 : Créer des ServiceAccounts

Créer `rbac-setup.yaml`

```
# Service Account pour Nginx

apiVersion: v1

kind: ServiceAccount

metadata:

  name: nginx-sa

  namespace: default

---

# Service Account pour Apache

apiVersion: v1

kind: ServiceAccount

metadata:

  name: apache-sa

  namespace: default

---

# Service Account pour MariaDB

apiVersion: v1

kind: ServiceAccount

metadata:

  name: mariadb-sa

  namespace: default
```

```
---

# Service Account pour un admin restreint

apiVersion: v1

kind: ServiceAccount

metadata:

  name: dev-user

  namespace: default
```

Appliquer

```
sudo k3s kubectl apply -f rbac-setup.yaml

# Vérifier

sudo k3s kubectl get sa
```

Étape 2 : Créer des Roles

Créer `rbac-roles.yaml`

```
# Role pour Nginx : lecture seule sur configmaps

apiVersion: rbac.authorization.k8s.io/v1

kind: Role

metadata:

  name: nginx-reader

spec:

  rules:

  - apiGroups: [""]

    resources: ["configmaps"]

    verbs: ["get", "list", "watch"]

---

# Role pour Dev : lecture des logs des pods

apiVersion: rbac.authorization.k8s.io/v1

kind: Role

metadata:

  name: dev-reader

spec:

  rules:
```

```

- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["events"]
  verbs: ["get", "list"]
---
# Role pour admin local : tout sauf delete
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: namespace-admin
spec:
  rules:
  - apiGroups: ["*"]
    resources: ["*"]
    verbs: ["get", "list", "watch", "create", "update", "patch"]

```

Appliquer

```

sudo k3s kubectl apply -f rbac-roles.yaml

# Vérifier
sudo k3s kubectl get roles

```

Étape 3 : Créer des RoleBindings

Créer `rbac-bindings.yaml`

```

# Bind nginx-reader à nginx-sa
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: nginx-reader-binding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role

```

```
    name: nginx-reader
subjects:
- kind: ServiceAccount
  name: nginx-sa
  namespace: default
---
# Bind dev-reader à dev-user
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-reader-binding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: dev-reader
subjects:
- kind: ServiceAccount
  name: dev-user
  namespace: default
---
# Bind namespace-admin à admin SA
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: admin-binding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: namespace-admin
subjects:
- kind: ServiceAccount
  name: admin-sa
```

```
namespace: default
```

Appliquer

```
sudo k3s kubectl apply -f rbac-bindings.yaml

# Vérifier

sudo k3s kubectl get rolebindings
```

Étape 4 : Assigner ServiceAccounts aux Pods

Éditer les Deployments

Pour chaque Deployment, ajouter :

```
spec:

  template:

    spec:

      serviceAccountName: nginx-sa # pour Nginx

      # ou mariadb-sa, apache-sa, etc.

      containers:

      - name: ...
```

Réappliquer les Deployments

```
sudo k3s kubectl apply -f nginx-deployment.yaml

sudo k3s kubectl apply -f apache-deployment.yaml

sudo k3s kubectl apply -f mariadb-deployment.yaml

# Vérifier

sudo k3s kubectl get pods -o jsonpath='{.items[*].spec.serviceAccountName}'
```

Étape 5 : Test des Permissions

Test 1 : Vérifier les permissions

```
# Récupérer le token du dev-user

TOKEN=$(sudo k3s kubectl create token dev-user)

# Tester l'accès avec le token

sudo k3s kubectl --token=$TOKEN get pods

# Doit fonctionner (lecture OK)
```

```
sudo k3s kubectl --token=$TOKEN delete pod <nom>

# Doit être refusé (delete non autorisé)
```

Test 2 : Vérifier les permissions du pod

```
# Depuis le pod Nginx

sudo k3s kubectl exec -it deployment/nginx -- \

    cat /var/run/secrets/kubernetes.io/serviceaccount/token

# Le token du pod est chiffré, avec ses permissions restreintes
```

Étape 6 : Sécurité Additionnelle

Pod Security Standards

```
apiVersion: policy/v1beta1

kind: PodSecurityPolicy

metadata:

  name: restricted

spec:

  privileged: false

  allowPrivilegeEscalation: false

  requiredDropCapabilities:

    - ALL

  runAsUser:

    rule: 'MustRunAsNonRoot'

  seLinux:

    rule: 'MustRunAs'

  fsGroup:

    rule: 'MustRunAs'

  readOnlyRootFilesystem: false
```

NetworkPolicy (Isoler le trafic)

```
apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

  name: app-network-policy

spec:
```



```
podSelector:
  matchLabels:
    app: nginx
policyTypes:
- Ingress
- Egress
ingress:
- from:
  - namespaceSelector: {}
  ports:
  - protocol: TCP
    port: 80
egress:
- to:
  - namespaceSelector: {}
  ports:
  - protocol: TCP
    port: 53 # DNS seulement
```

Vérification Complète (JOB 08)

```
# 1. ServiceAccounts
sudo k3s kubectl get sa

# 2. Roles
sudo k3s kubectl get roles

# 3. RoleBindings
sudo k3s kubectl get rolebindings

# 4. Vérifier que les pods utilisent les SA
sudo k3s kubectl get pods -o jsonpath='{.items[*].spec.serviceAccountName}'

# 5. Audit : qui peut faire quoi?
```

```
sudo k3s kubectl auth can-i get pods
--as=system:serviceaccount:default:dev-user

# Output: yes

sudo k3s kubectl auth can-i delete pods
--as=system:serviceaccount:default:dev-user

# Output: no
```

Dépannage JOB 08

Problème	Solution
`Error: resourcequotas is forbidden`	Permissions RBAC insuffisantes
Token invalide	Créer un nouveau token: `kubectl create token <sa>`
Pod ne démarre pas	ServiceAccount non trouvé dans le namespace

✓ Prêt pour JOB 09 ?

```
# Vérifier que RBAC est en place

sudo k3s kubectl get sa,roles,rolebindings | wc -l

# Doit afficher > 10 objets
```

→ Suivant : JOB 09 — Helm & Automatisation

Notes de l'étudiant :

```
- ServiceAccounts créés: ☐

- Roles assignés: ☐

- RoleBindings en place: ☐

- Permissions testées: ☐
```

JOB 09 — Helm & Industrialisation

Objectif : Empaqueter et déployer une application complète via Helm. Maîtriser install, upgrade, rollback.

Durée estimée : 25 minutes

Prérequis : JOB 08 complété

Helm : Concepts

Objet	Rôle
Chart	Paquet d'une application (templates + values)
Release	Instance installée d'un chart
Values	Paramètres personnalisables (values.yaml)
Template	Manifeste K8s avec variables {{ }}
Repository	Dépôt de charts (public ou privé)

Étape 1 : Installer Helm

```
# Télécharger Helm

curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |
bash

# Vérifier

helm version

# Ajouter un dépôt public

helm repo add bitnami https://charts.bitnami.com/bitnami

helm repo update

# Lister les repos

helm repo list
```

Étape 2 : Créer un Chart Personnalisé

Créer la structure

```
helm create k3s-app

cd k3s-app
```

Cela génère :

```
k3s-app/
├─ Chart.yaml
```

```
|— values.yaml
|— templates/
|   |— deployment.yaml
|   |— service.yaml
|   |— configmap.yaml
|   └─ _helpers.tpl
└─ charts/
```

Éditer `Chart.yaml`

```
apiVersion: v2

name: k3s-app

description: Application K3S multi-composants

type: application

version: 1.0.0

appVersion: "1.0"

keywords:

  - k3s

  - nginx

  - apache

  - mariadb

maintainers:

  - name: Étudiant

    email: etudiant@example.com
```

Éditer `values.yaml`

```
# Nginx

nginx:

  enabled: true

  replicaCount: 3

  image:

    repository: nginx

    tag: latest

  port: 80

  nodePort: 30080

  resources:
```

```
    requests:
      cpu: 100m
      memory: 128Mi

# Apache
apache:
  enabled: true
  replicaCount: 2
  image:
    repository: httpd
    tag: latest
  port: 80
  nodePort: 30081
  resources:
    requests:
      cpu: 100m
      memory: 128Mi

# MariaDB
mariadb:
  enabled: true
  replicaCount: 1
  image:
    repository: mariadb
    tag: latest
  port: 3306
  rootPassword: "SecurePass123!"
  resources:
    requests:
      cpu: 250m
      memory: 512Mi
```

```
# Global

global:

  environment: production
```

Étape 3 : Créer les Templates Helm

`templates/deployment.yaml`

```
{{- if .Values.nginx.enabled }}

apiVersion: apps/v1

kind: Deployment

metadata:

  name: {{ include "k3s-app.fullname" . }}-nginx

  labels:

    {{- include "k3s-app.labels" . | nindent 4 }}

    app: nginx

spec:

  replicas: {{ .Values.nginx.replicaCount }}

  selector:

    matchLabels:

      {{- include "k3s-app.selectorLabels" . | nindent 6 }}

      app: nginx

  template:

    metadata:

      labels:

        {{- include "k3s-app.selectorLabels" . | nindent 8 }}

        app: nginx

    spec:

      containers:

        - name: nginx

          image: "{{ .Values.nginx.image.repository }}:{{
.Values.nginx.image.tag }}"

          ports:

            - containerPort: {{ .Values.nginx.port }}

          resources:

            {{- toYaml .Values.nginx.resources | nindent 12 }}
```

```

{{- end }}

---

{{- if .Values.apache.enabled }}

apiVersion: apps/v1

kind: Deployment

metadata:

  name: {{ include "k3s-app.fullname" . }}-apache

spec:

  replicas: {{ .Values.apache.replicaCount }}

  # ... similaire à Nginx ...

{{- end }}

```

`templates/service.yaml`

```

{{- if .Values.nginx.enabled }}

apiVersion: v1

kind: Service

metadata:

  name: {{ include "k3s-app.fullname" . }}-nginx

spec:

  type: NodePort

  selector:

    {{- include "k3s-app.selectorLabels" . | nindent 4 }}

    app: nginx

  ports:

    - port: {{ .Values.nginx.port }}

      targetPort: {{ .Values.nginx.port }}

      nodePort: {{ .Values.nginx.nodePort }}

{{- end }}

```

Étape 4 : Installer le Chart

Valider le chart

```

helm lint k3s-app/

# Doit afficher: 1 chart(s) linted, 0 error(s)

```

Installer en dry-run (test)

```
helm install k3s-release k3s-app/ --dry-run --debug

# Affiche les manifests sans les appliquer
```

Installer pour de vrai

```
helm install k3s-release k3s-app/

# Vérifier

helm list

kubectl get all
```

Étape 5 : Mettre à Jour le Chart (Upgrade)

Changer une valeur

```
# Augmenter les replicas Nginx

helm upgrade k3s-release k3s-app/ \
  --set nginx.replicaCount=5

# K3S mettra à jour progressivement

kubectl get pods | grep nginx

# Doit afficher 5 pods
```

Voir l'historique des releases

```
helm history k3s-release

# Output:

# REVISION  UPDATED              STATUS          CHART          DESCRIPTION
# 1         ...              DEPLOYED       k3s-app-1.0    Install complete
# 2         ...              DEPLOYED       k3s-app-1.0    Upgrade complete
```

Étape 6 : Rollback

Revenir à la version précédente

```
# Revenir à la révision 1

helm rollback k3s-release 1

# Vérifier

helm list
```



```
kubect1 get pods | grep nginx

# Doit afficher 3 pods à nouveau
```

Étape 7 : Installer un Chart Public

Chercher un chart

```
helm search repo bitnami | grep -i nginx
```

Installer un chart public

```
# Installer Nginx depuis Bitnami

helm install my-nginx bitnami/nginx \

  --set replicaCount=3 \

  --set service.type=NodePort \

  --set service.nodePort=30090


# Vérifier

helm list

kubect1 get pods

curl http://192.168.1.11:30090
```

Personnaliser avec un fichier values

```
# Créer my-values.yaml

cat > my-values.yaml << EOF

replicaCount: 5

image:

  tag: "1.25"

service:

  type: NodePort

  nodePort: 30090

EOF


# Installer avec les paramètres personnalisés

helm install my-nginx bitnami/nginx -f my-values.yaml
```

Vérification Complète (JOB 09)

```
# 1. Helm installé?

helm version
```

```
# 2. Charts disponibles?

helm search repo bitnami | head -5


# 3. Releases installées?

helm list -a


# 4. Vérifier un chart personnel

helm get values k3s-release


# 5. Voir les manifests générés

helm get manifest k3s-release | head -30


# 6. Vérifier l'historique

helm history k3s-release


# 7. Pods en place?

kubectl get pods | grep -E 'nginx|apache|mariadb'
```

Étape 8 : Supprimer une Release

```
# Supprimer la release

helm uninstall k3s-release


# Vérifier

helm list

kubectl get pods | grep -E 'nginx|apache'

# Doit être vide
```

Étape 9 : Package et Publish (Bonus)

Packager le chart

```
helm package k3s-app/


# Génère k3s-app-1.0.0.tgz
```

Installer depuis le package

```
helm install k3s-release k3s-app-1.0.0.tgz
```

Publier dans un dépôt (avancé)

Voir la documentation Helm pour HeartBeat ou Artifact Hub.

Dépannage JOB 09

Problème	Solution
`helm: not found`	Réinstaller Helm (voir Étape 1)
Template error	Vérifier la syntaxe Helm (`helm lint`)
Values pas appliquées	Vérifier les indentations YAML
Rollback échoue	Révision invalide → `helm history`

✓ Validation Finale

```
# Tous les JOBS complétés?

helm list

kubectl get nodes

kubectl get pods -A | wc -l

# Doit afficher > 20 pods
```

→ Prêt pour la présentation!

Ressources

- Helm Docs : <https://helm.sh/docs/>
- Helm Chart Template Guide : https://helm.sh/docs/chart_template_guide/
- Artifact Hub : <https://artifacthub.io/>

Notes de l'étudiant :

```
- Helm installé: ☐

- Chart personnel créé: ☐

- Install + Upgrade + Rollback testés: ☐

- Chart public installé: ☐

- Release supprimée: ☐
```

JOB 10 — Comparaison K3S vs Docker vs Docker Swarm

Objectif

Comparer trois technologies d'orchestration de conteneurs : K3S, Docker (standalone) et Docker Swarm.

Vue d'ensemble

Aspect	Docker	Docker Swarm	K3S / Kubernetes
Type	Moteur de conteneurs	Orchestration légère	Orchestration complète
Taille	~200 MB	~100 MB	~10 MB (K3S)
Learning Curve	Facile	Moyen	Steepe
Scalabilité	Faible (1 hôte)	Moyenne (100+ nodes)	Très élevée (5000+ nodes)
Production ready	✓ (simple apps)	✓ (équipes expertes)	✓ ✓ (recommandé)
Cas d'usage	Dev, test, apps simples	PME, équipes petites	Entreprises, cloud
Communauté	Énorme	Petite	Énorme
Alternatives	-	Nomad, Mesos	ECS, OpenShift

Docker (Standalone)

Qu'est-ce que c'est ?

Docker est un moteur pour exécuter des conteneurs sur un seul hôte ou manuellement sur plusieurs.

Architecture



Avantages

- ✓ Simple à apprendre et mettre en place
- ✓ Parfait pour le développement
- ✓ Léger (~200 MB)
- ✓ Exécution rapide
- ✓ Excellent community support

Inconvénients

- ✗ Pas de scaling automatique
- ✗ Pas de failover automatique

- ❌ Pas de load balancing intégré
- ❌ Gestion manuelle de plusieurs hôtes
- ❌ Pas de rollback automatique

Exemple : Lancer un Nginx

```
docker run -d -p 80:80 --name web nginx:latest

# Chaque conteneur est isolé

# Les données se perdent au redémarrage
```

Cas d'usage

- Développement local
- Tests unitaires
- Prototypes
- Apps monolithiques simples
- Conteneurs temporaires

Docker Swarm

Qu'est-ce que c'est ?

Docker Swarm est la solution native de Docker pour l'orchestration sur plusieurs hôtes.

Architecture

```
Manager (Swarm Leader)
├─ etcd (state store)
├─ Scheduler
└─ Orchestrator

Worker 1      Worker 2      Worker 3
├─ Docker     ── Docker     ── Docker
└─ Tasks      ── Tasks      ── Tasks
```

Concepts clés

- Service : Application (comme deployment K8s)
- Task : Conteneur en cours d'exécution
- Manager : Pilote le cluster
- Worker : Exécute les conteneurs

Avantages

- ✅ Plus simple que Kubernetes
- ✅ Intégré à Docker

- ✓ Faible overhead
- ✓ Facile de passer de Docker à Swarm
- ✓ Bon pour petits clusters (< 100 nodes)

Inconvénients

- ✗ Pas de résilience du control plane (1 manager)
- ✗ Capacités limitées comparé à K8s
- ✗ Peu de features avancées
- ✗ Communauté petite
- ✗ Maintenance moins active

Exemple : Créer un service

```
# Initialiser Swarm

docker swarm init

# Créer un service avec 3 replicas

docker service create \

  --name web \

  --replicas 3 \

  -p 80:80 \

  nginx:latest

# Docker orchestre automatiquement

# Failover automatique si un conteneur crash
```

Cas d'usage

- Petits clusters (< 10 nodes)
- Équipes avec expertise Docker
- Apps moyennes sans complexité haute
- Organisations utilisant déjà Docker
- ✗ Pas recommandé pour production critique

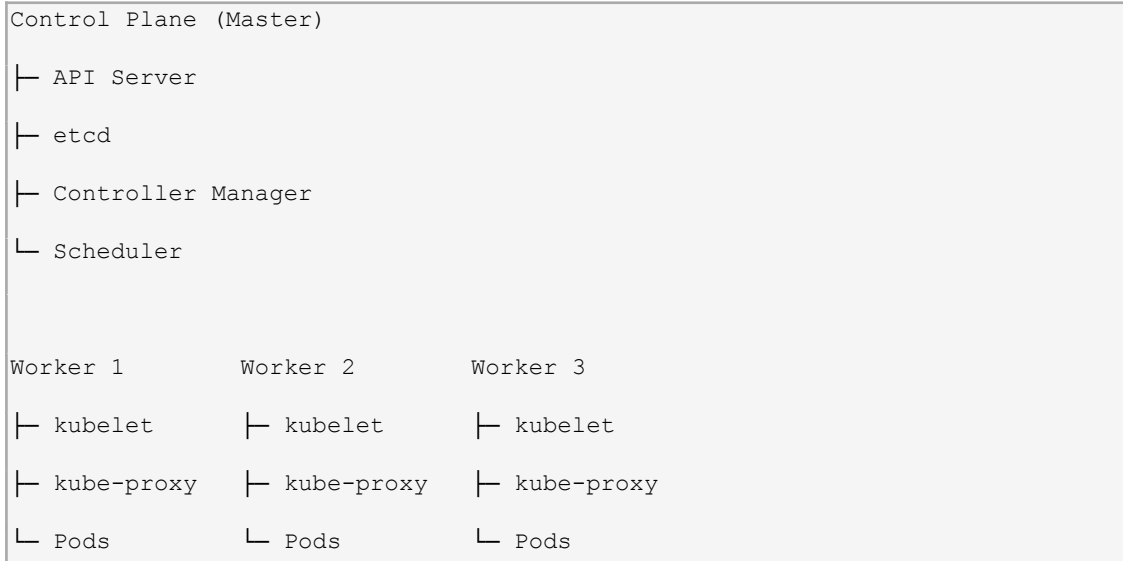
Kubernetes / K3S

Qu'est-ce que c'est ?

Kubernetes est un orchestrateur d'entreprise pour clusters de toutes tailles.

K3S est une version allégée (10 MB vs 1.3 GB).

Architecture



Concepts clés

- Pod : Unité d'exécution (1+ conteneurs)
- Deployment : Gestion des replicas
- Service : Exposition des pods
- Volume : Stockage persistant
- ConfigMap : Configuration
- Secret : Données sensibles
- RBAC : Contrôle d'accès

Avantages

- ✓ Standard de l'industrie
- ✓ Hautement scalable (5000+ nodes)
- ✓ Features très avancées
- ✓ Excellent ecosystem
- ✓ Multi-cloud
- ✓ Production-ready
- ✓ Automation et GitOps

Inconvénients

- ✗ Complexe à apprendre
- ✗ Overhead de ressources
- ✗ Courbe d'apprentissage steepe
- ✗ Nécessite expertise
- ✗ Plus lent à mettre en place

Exemple : Créer un Deployment

```
kubectl create deployment web --image=nginx:latest

kubectl scale deployment web --replicas=3

kubectl expose deployment web --port=80

# Kubernetes orchestre complètement

# Failover, scaling, mises à jour rolling, etc.
```

Cas d'usage

- Production enterprise
- Microservices complexes
- Multi-cloud / hybrid
- High availability critique
- Équipes DevOps
- Applications scale-out

Tableau comparatif détaillé

Caractéristiques techniques

Feature	Docker	Swarm	K3S	K8s
Orchestration	✗	✓	✓✓	✓✓
HA Control Plane	✗	⚠ (1 manager)	✓	✓
Scaling	Manuel	Semi-auto	Auto	Auto
Rolling Updates	Manuel	✓	✓	✓
Rollback	Manuel	✓	✓	✓
Health Checks	✗	✓	✓	✓
Load Balancing	✗	✓	✓	✓
Storage	Volumes	Volumes	PV/PVC	PV/PVC
Networking	Overlay	Overlay	CNI	CNI
RBAC	✗	⚠	✓	✓
Monitoring	Externe	Externe	Compatible	Compatible
Logging	Externe	Externe	Compatible	Compatible

Performance et ressources

Métrique	Docker	Swarm	K3S	K8s
Taille	200 MB	100 MB	10 MB	1.3 GB
RAM min	512 MB	1 GB	512 MB	2 GB
CPU min	1 cœur	1 cœur	1 cœur	2 cœurs
Startup	< 10s	30s	30s	1-2m
Max nodes	1	100+	100+	5000+

Latence API	-	~50ms	~100ms	~200ms
------------------------	---	-------	--------	--------

Coûts et complexité

Aspect	Docker	Swarm	K3S	K8s
Setup	Très simple	Simple	Moyen	Complexe
Maintenance	Manuelle	Manuelle	Auto	Auto
Expertise	Junior	Intermédiaire	Intermédiaire	Senior
Documentation	Excellente	Bonne	Excellente	Excellente
Coût d'infrastructure	Très faible	Faible	Faible	Moyen
Coût d'expertise	Bas	Moyen	Moyen	Élevé

Quelle technologie choisir ?

Choisir Docker Standalone si...

- Vous travaillez seul ou en petit groupe
- Vous avez 1-2 hôtes seulement
- Vous testez ou développez
- L'infrastructure est simple
- Vous ne besoin pas de HA
- Budget très limité

Exemple : Startup avec 1 serveur

Choisir Docker Swarm si...

- Vous avez 5-50 nœuds
- Vous utilisez déjà Docker
- L'équipe maîtrise Docker
- Vous voulez simplicité > fonctionnalités
- Budget faible à moyen
- Applications peu critiques

Exemple : PME avec 10 serveurs

Choisir K3S si...

- Vous voulez Kubernetes sur ressources limitées
- IoT, edge computing, labs
- Vous apprenez Kubernetes
- Budget moyen, équipe réduite
- Environnements embarqués

Exemple : Environnement de lab, IoT edge

Choisir Kubernetes complet si...

- Vous avez 50+ nœuds
- Mission-critical, haute disponibilité exigée
- Besoin de fonctionnalités avancées
- Multi-cloud / hybrid cloud
- Équipe DevOps expérimentée
- Budget important

Exemple : Grand groupe, cloud public

Migration entre technologies

Docker → Docker Swarm

```
# Simple, syntaxe similaire

# Fichiers docker-compose → docker service

docker-compose up          → docker stack deploy
```

Docker Swarm → K3S/Kubernetes

```
# Plus complexe

# Services → Deployments

# Networks → Services

# Volumes → PersistentVolumes

# Nécessite refactoring du code
```

Kubernetes → Autre Kubernetes

```
# Facile entre distributions

# K3S → EKS (AWS)

# K3S → GKE (Google)

# Manifestos K8s compatibles
```

Croissance et évolution

Scénario typique

```
Phase 1: Développement

└─ Docker standalone

    1 serveur, simple

Phase 2: Première production

└─ Docker Swarm ou K3S

    5-10 serveurs, HA basique
```

Phase 3: Croissance

└ K3S ou Kubernetes

50+ serveurs, microservices

Phase 4: Enterprise

└ Kubernetes multicluster

100+ serveurs, multi-régions

Tableau récapitulatif : Avantages vs Inconvénients

Docker Standalone

✓	✗
Très simple	Pas d'orchestration
Léger	Pas de HA
Rapide à démarrer	Scaling manuel
Parfait pour dev	Pas de failover
Community énorme	Pas de load balance auto

Docker Swarm

✓	✗
Plus simple que K8s	Moins puissant que K8s
Intégré à Docker	Communauté petite
Faible overhead	Features limitées
Facile pour petits clusters	Pas de HA du control plane
Moins expertise nécessaire	Maintenace faible

K3S / Kubernetes

✓	✗
Standard industrie	Complexe
Très puissant	Overhead de ressources
HA complète	Courbe d'apprentissage
Scaling automatique	Nécessite expertise
Ecosystème énorme	Setup plus long
Multi-cloud	Coût d'expertise

Recommandations finales

Pour un développeur

Docker standalone pour apprendre et tester

Pour une PME

Docker Swarm pour petits clusters

K3S pour futureproofing

Pour une entreprise

Kubernetes (K3S ou managed) pour production

Pour IoT / Edge

K3S pour légèreté + puissance

Pour apprendre

K3S car c'est Kubernetes miniature

Compétences transférables à K8s complet

Ressources pour aller plus loin

Docker

- <https://docs.docker.com/>
- Docker official documentation

Docker Swarm

- <https://docs.docker.com/engine/swarm/>
- Swarm mode

Kubernetes / K3S

- <https://kubernetes.io/docs/>
- <https://k3s.io/>
- Kubernetes official documentation

Certifications

- CKA (Certified Kubernetes Administrator)
- CKAD (Certified Kubernetes Application Developer)
- Docker Certified Associate

Conclusion

Technologie	Score Simplicité	Score Puissance	Recommandé pour
Docker	★★★★★	★	Dev, tests
Docker Swarm	★★★★★	★★★★	Petits clusters
K3S	★★★★	★★★★★★★★	Production
Kubernetes	★★★	★★★★★★★★	Enterprise

Choix de l'étudiant pour ce projet : K3S car :

- ☒ Allie simplicité et puissance
 - ☒ Vrai Kubernetes en miniature
 - ☒ Compétences transférables
 - ☒ Production-ready
 - ☒ Parfait pour apprendre
- ☒ Comparaison complète ! Vous êtes maintenant expert en orchestration de conteneurs !

Annexes — Commandes, Dépannage et Memos

Table des matières

- Mémo kubectl
- Mémo Kubernetes
- Mémo Helm
- Dépannage courant
- Debugging avancé
- Performance et monitoring

Mémo kubectl

Installation et configuration

```
# Installer kubectl

sudo apt install -y kubectl

# Ou avec Kubernetes

curl -O https://dl.k8s.io/release/v1.28.0/bin/linux/amd64/kubectl

sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# Vérifier

kubectl version

kubectl cluster-info

kubectl config current-context
```

Contextes et clusters

```
# Voir les contextes

kubectl config get-contexts

kubectl config current-context

# Changer de contexte

kubectl config use-context my-context

# Voir le kubeconfig

cat ~/.kube/config

# Merger des kubeconfigs
```

```
KUBECONFIG=~/.kube/config:~/.kube/other-config kubectl config view
```

Navigation et exploration

```
# Clusters

kubectl cluster-info

kubectl get nodes

kubectl get nodes -o wide

kubectl describe node kubes-01


# Namespaces

kubectl get namespaces

kubectl create namespace my-ns

kubectl delete namespace my-ns

kubectl config set-context --current --namespace=my-ns


# Ressources

kubectl api-resources

kubectl api-versions

kubectl explain pods      # Doc d'une ressource
```

Déploiement

```
# Appliquer des manifests

kubectl apply -f deployment.yaml

kubectl apply -f *.yaml

kubectl apply -R -f ./manifests/


# Créer rapidement

kubectl create deployment my-app --image=nginx:latest

kubectl create service nodeport web --tcp=8080:80


# Voir les deployments

kubectl get deployments

kubectl get deployment my-app -o yaml
```

Pods et exécution

```
# Voir les pods
```

```
kubectl get pods

kubectl get pods -n kube-system

kubectl get pods -A (--all-namespaces)

kubectl get pods -o wide

kubectl get pods -l app=nginx


# Détails

kubectl describe pod my-pod

kubectl get pod my-pod -o yaml


# Logs

kubectl logs my-pod

kubectl logs -f deployment/my-app

kubectl logs --all-containers=true my-pod

kubectl logs my-pod --previous


# Exécuter

kubectl exec -it my-pod -- /bin/bash

kubectl exec my-pod -- ls -la

kubectl attach my-pod -i -t
```

Services et networking

```
# Voir les services

kubectl get services

kubectl get svc

kubectl describe svc my-service


# Port forwarding

kubectl port-forward pod/my-pod 8080:80

kubectl port-forward svc/my-service 8080:80


# Tester la connectivité

kubectl exec my-pod -- curl http://my-service:80
```


Modifications et mises à jour

```
# Éditer une ressource

kubectl edit deployment my-app

kubectl edit pod my-pod


# Patcher

kubectl patch deployment my-app -p '{"spec":{"replicas":5}}'

kubectl patch service my-svc -p '{"spec":{"type":"LoadBalancer"}}'


# Set

kubectl set image deployment/my-app app=nginx:1.25

kubectl set env deployment/my-app ENV_VAR=value


# Scale

kubectl scale deployment my-app --replicas=5
```

Suppression

```
# Supprimer une ressource

kubectl delete pod my-pod

kubectl delete deployment my-app

kubectl delete -f deployment.yaml


# Suppression immédiate

kubectl delete pod my-pod --grace-period=0 --force
```

Troubleshooting

```
# État général

kubectl get events

kubectl get events -A

kubectl top nodes

kubectl top pods


# Diagnostics

kubectl describe nodes

kubectl describe pods
```

```
kubectl get pod my-pod -o yaml | grep -i status
```

```
# Logs système
```

```
kubectl logs -n kube-system [POD_NAME]
```

Mémo Kubernetes

Ressources principales

```
# Pod (unité de base)
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: my-pod
```

```
spec:
```

```
  containers:
```

```
    - name: my-container
```

```
      image: nginx:latest
```

```
# Deployment (avec replicas)
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: my-app
```

```
spec:
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: my-app
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: my-app
```

```
    spec:
```

```
      containers:
```

```
- name: app
  image: myapp:latest

# Service (expose les pods)
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: LoadBalancer
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080

# ConfigMap (configuration)
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  config.yml: |
    server:
      port: 8080

# Secret (données sensibles)
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
```

```
stringData:
  password: supersecret

# PersistentVolume (stockage)
apiVersion: v1
kind: PersistentVolume
metadata:
  name: my-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /data

# PersistentVolumeClaim
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi

# Namespace
apiVersion: v1
kind: Namespace
metadata:
```

```
name: my-namespace
```

Labels et sélecteurs

```
# Ajouter un label

kubectl label pod my-pod app=nginx


# Filtrer par label

kubectl get pods -l app=nginx

kubectl get pods -l env=production,tier=frontend


# Supprimer un label

kubectl label pod my-pod app-
```

Probes (santé des pods)

```
apiVersion: v1

kind: Pod

metadata:

  name: my-pod

spec:

  containers:

    - name: app

      image: myapp:latest


      # Liveness probe (redémarrer si failed)

      livenessProbe:

        httpGet:

          path: /healthz

          port: 8080

          initialDelaySeconds: 15

          periodSeconds: 20


      # Readiness probe (trafic si ready)

      readinessProbe:

        httpGet:

          path: /ready
```

```
    port: 8080

    initialDelaySeconds: 5

    periodSeconds: 10

# Startup probe (appli lent démarrage)

startupProbe:

  httpGet:

    path: /startup

    port: 8080

  failureThreshold: 30

  periodSeconds: 10
```

Mémo Helm

Installation et setup

```
# Installer Helm

curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |
bash

# Vérifier

helm version

helm list

# Ajouter un repository

helm repo add bitnami https://charts.bitnami.com/bitnami

helm repo update

helm search repo nginx
```

Charts

```
# Créer un chart

helm create my-chart

# Voir la structure

tree my-chart/

# Valider
```

```
helm lint my-chart

# Voir les templates générés

helm template my-release my-chart

helm template my-release my-chart --values values.yaml
```

Installation et management

```
# Installer une release

helm install my-release my-chart

# Avec valeurs personnalisées

helm install my-release my-chart \
  --values custom-values.yaml \
  --set replicaCount=5

# Voir les releases

helm list

helm list -a (--all)

helm list -n my-namespace

# Statut

helm status my-release

helm get values my-release

helm get notes my-release

helm get manifest my-release
```

Mises à jour et rollback

```
# Mettre à jour

helm upgrade my-release my-chart

helm upgrade my-release my-chart --values new-values.yaml

# Historique

helm history my-release

# Rollback
```

```
helm rollback my-release 1

helm rollback my-release 2


# Supprimer

helm uninstall my-release

helm uninstall my-release --keep-history
```

Dépannage courant

Problème : Pod reste en Pending

```
# Vérifier l'événement

kubectl describe pod my-pod

# Regarder la section "Events"


# Causes possibles :

# - Pas assez de ressources (CPU/RAM)

# - PVC non liée

# - Image non trouvée

# - Pas de nœud compatible


# Solutions :

kubectl get nodes      # Voir l'espace disponible

kubectl top nodes      # Voir l'utilisation

kubectl scale deployment my-app --replicas=1 # Réduire
```

Problème : Pod en CrashLoopBackOff

```
# Voir les logs

kubectl logs my-pod

kubectl logs my-pod --previous


# Vérifier la configuration

kubectl describe pod my-pod


# Problèmes courants :

# - Application ne démarre pas
```



```
# - Fichiers de config manquants  
# - Permissions insuffisantes  
# - Ports déjà occupés
```

Problème : Pod ne se connecte pas au service

```
# Tester la DNS  
  
kubectl exec my-pod -- nslookup my-service  
  
kubectl exec my-pod -- nslookup my-service.default.svc.cluster.local  
  
# Tester la connectivité  
  
kubectl exec my-pod -- curl http://my-service  
  
# Vérifier le service  
  
kubectl get svc  
  
kubectl describe svc my-service  
  
# Vérifier les endpoints  
  
kubectl get endpoints my-service
```

Problème : Image ne se charge pas

```
# Vérifier l'image existe  
  
docker pull nginx:latest # Tester localement  
  
# Voir les événements  
  
kubectl describe pod my-pod  
  
# Vérifier le registre  
  
kubectl get secret -n default  
  
# Ajouter les credentials du registre  
  
kubectl create secret docker-registry my-secret \  
  --docker-server=registry.example.com \  
  --docker-username=user \  
  --docker-password=pass
```

Problème : Pas d'accès au stockage persistant

```
# Vérifier les PV/PVC

kubectl get pv

kubectl get pvc

kubectl describe pvc my-pvc


# Vérifier les montages

kubectl exec my-pod -- mount | grep -i storage


# Voir l'espace disque

kubectl exec my-pod -- df -h
```

Problème : Secret/ConfigMap non mis à jour

```
# Les pods ne reloadent pas automatiquement !

# Solution 1 : Redéployer les pods

kubectl rollout restart deployment/my-app


# Solution 2 : Utiliser une image avec watch

# (l'app relit les fichiers)


# Vérifier le secret/configmap

kubectl get secret my-secret -o yaml

kubectl get configmap my-config -o yaml
```

Debugging avancé

Traçage des requêtes API

```
# Mode verbose

kubectl get pods -v 9


# Voir les appels API

kubectl get pods -v 8 2>&1 | grep -i "GET"


# Audit des événements

kubectl get events -A --sort-by='.lastTimestamp'
```

Inspection approfondie

```
# État détaillé d'un pod

kubectl get pod my-pod -o json | jq '.status'

# Voir tous les labels

kubectl get pod my-pod --show-labels

# Voir les annotations

kubectl get pod my-pod -o jsonpath='{.metadata.annotations}' | jq .

# Voir les variables d'environnement

kubectl exec my-pod -- env
```

Network troubleshooting

```
# Faire un ping entre pods

kubectl exec my-pod -- ping -c 2 other-pod

# Tester DNS

kubectl exec my-pod -- nslookup kubernetes.default

# Lancer un pod de debug

kubectl run -it debug --image=busybox -- sh

# Ou

kubectl debug -it pod/my-pod --image=busybox
```

Accès aux fichiers

```
# Copier un fichier depuis le pod

kubectl cp default/my-pod:/var/log/app.log ./app.log

# Copier vers le pod

kubectl cp ./config.yml default/my-pod:/etc/app/config.yml

# Voir le filesystem

kubectl exec my-pod -- ls -la /

kubectl exec my-pod -- find / -name "config.*"
```

Performance et monitoring

Ressources utilisées

```
# Usage des nœuds

kubectl top nodes

kubectl top nodes -l kubernetes.io/hostname=kubes-01


# Usage des pods

kubectl top pods

kubectl top pods -n kube-system

kubectl top pods -A


# Voir les limits et requests

kubectl describe nodes

kubectl describe pod my-pod | grep -A 5 "Requests\\|Limits"
```

Monitoring simple

```
# Watch les pods

kubectl get pods --watch


# Watch les events

kubectl get events -A --watch


# Watch les métriques

watch kubectl top nodes

watch kubectl top pods
```

Logs en temps réel

```
# Tous les logs

kubectl logs deployment/my-app --all-containers=true -f


# Plusieurs pods

kubectl logs -l app=nginx -f


# Pour un namespace
```

```
kubectl logs -n kube-system -l component=kubelet -f
```

Checklists rapides

Checklist : Pod santé

```
[ ] kubectl get pods                # Status Running
[ ] kubectl describe pod my-pod      # No events errors
[ ] kubectl logs my-pod              # No errors in logs
[ ] kubectl exec my-pod -- curl localhost # App répond
[ ] kubectl top pod my-pod           # Ressources OK
```

Checklist : Deployment santé

```
[ ] kubectl get deployments          # READY = DESIRED
[ ] kubectl rollout status deployment/my-app
[ ] kubectl get rs                   # Proper number of replicas
[ ] kubectl get pods -l app=my-app   # All running
[ ] kubectl get svc my-service       # Has endpoints
[ ] curl http://my-service           # Service répond
```

Checklist : Cluster santé

```
[ ] kubectl get nodes                # All Ready
[ ] kubectl get pods -n kube-system  # All Running
[ ] kubectl api-resources             # API responsive
[ ] kubectl get events -A            # No alarming events
[ ] kubectl top nodes                # Resources OK
```

Fichiers de configuration utiles

kubeconfig pour accès distant

```
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: [cert]
    server: https://kubes-01.local:6443
    name: k3s-cluster
contexts:
- context:
    cluster: k3s-cluster
```

```
    user: alice

  name: alice-context
current-context: alice-context
kind: Config
preferences: {}
users:
- name: alice

  user:

    client-certificate-data: [cert]

    client-key-data: [key]
```

NetworkPolicy *basique*

```
apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

  name: deny-all

spec:

  podSelector: {}

  policyTypes:

    - Ingress

  ingress:

    - from:

        - podSelector:

            matchLabels:

              app: my-app
```

Commandes d'urgence

```
# Redémarrer rapidement

kubectl rollout restart deployment/my-app


# Arrêter une app

kubectl scale deployment my-app --replicas=0


# Lancer rapidement
```

```
kubectl scale deployment my-app --replicas=3

# Supprimer un pod coincé
kubectl delete pod my-pod --grace-period=0 --force

# Voir tous les erreurs
kubectl get events -A --sort-by='.lastTimestamp' | grep -i error

# Drain un nœud
kubectl drain kubes-02 --ignore-daemonsets

# Uncordon un nœud
kubectl uncordon kubes-02
```

Prochaines étapes

Consultez ces ressources si vous êtes bloqué :

- [Documentation Kubernetes officielle](#)
- [K3S documentation](#)
- [Helm documentation](#)



Vous avez toutes les commandes et solutions courantes !